#Recap

1. Given list of numbers, find the minimum value using the built in function "min".

```
For example: l = [38, 76, 21, 90, -3, 99]
Answer: -3
```

```
l = [38, 76, 21, 90, -3, 99]
#write your code here
min(l) #built in function

-3
```

1. Find the minimum value using custom loop.

```
#write your code here:
l = [38, 76, 21, 90, -3, 99]
'''
we loop from the first value to the end step by step
1. minimum = 38
2. 76...checks if 76 is below the minimum.. no.. so we keep38
3. 21...checks if 21 is belwo the minimu.. 21 < 38, yes
    minimum = 21
4. 90... check if 90 is below theminimum.. 90 < 21, no
5. -3.. check if -3 is beow the minimum.. -3 < 21, yes
    minimum = -3
6. 99... check if 99 is below the minimum.. 99 < -3, no

minimum =3
'''
minimum = l[0] #we take the firs value as minimum initially
for value in l: # take each itme one by one
  if value < minimum:
    minimum = value #update teh minimum

print(minimum)

-3
```

1. Write a function named `minimum` that takes list as an argument and returns the minimum value from the list.

for example:

```
l = [38, 76, 21, 90, -3, 99]
minimum(l)
```

Returns: -3

```
def minimum(myList): #user-defined function
  #write your code here
  #it returns the minimum value in the list
  minimum = myList[0]
  for value in myList:
    if value < minimum:
      minimum = value

  return minimum

Sema = [389, -122, 8, -989, 643 , 90, -99]
minimum(Sema)

-989

Kebron = [89, 37, 9, 87]
minimum(Kebron)

9

# we want to find hte index of the minimum value
l = [38, 76, -21, 90, -3, 99]
#let's find the minimum first
minimum = 0
for value in range(len(l)): #range(6), 0, 1, 2...5
  if l[value] < l[minimum]:
    minimum = value
print(minimum)


2
```

1. Do minor tweak to your previous code and change the function to return the maximum value instead of minimum.

```
l = [38, 76, 21, 90, -3, 99]
miximum(l)
Answer: 99

def maximum(l):
  pass
  #write your code here
```

1. Given the names and grades for each student in a class of students, store them in a nested list and print the name(s) of any student(s) having the lowest grade.

Note: If there are multiple students with the lowest grade, order their names alphabetically and print each name on a new line.

Example:

The ordered list of scores is records = `[["chi", 20], ["alpha", 20], ["Beta", 50]]`, so the lowest score is 20. There are two students with that score "chi" & "alpha"

Ordered alphabetically, the names are printed as:

```
alpha
chi

records = [["chi", 20], ["alpha", 20], ["Beta", 50]]
#write your code here
```

1. Imagine we have the following list

`fruits = ['orange', 'banana', 'apple', 'pineapple']`

What if we want to print only orange from the list. which one of the following two codes work and reason out why?

Solution1:

```
fruits = ['orange', 'banana', 'apple', 'pineapple']
i = 0
while i < len(fruits):
  print(fruits[i])
  if i > 0: #what is wrong here?
    break
  i+=1
```

Solution 2:

```
fruits = ['orange', 'banana', 'apple', 'pineapple']
i = 0
while i < len(fruits):
  print(fruits[i])
  i+=1
  if i > 0:
    break

#check the codes here:
```

Types of Arguments

1. Arbitrary Arguments (*args)

- when we do not know how many arguments that will be passed into our function
- add a * before the parameter name in the function definition.

- This way the function will receive a tuple of arguments, and can access the items accordingly

```
def add(a, b):
  return a + b

add(4, 2)
```

```
6
```

```
def adder(*a):
  for i in a:
    print(i)

adder(1, 90, 100, 101)
```

```
1
90
100
101
```

```
adder(1, 90, 2)
```

```
1
90
2
```

```
adder(2)
```

```
2
```

```
def my_function(*kids):
  print("The youngest child is " + kids[2]) # kids[2] mean from the
arguemnts in kids it will take the value at 2nd index

my_function("Emil", "Natalie", "Linus",'William') # the value at
second index is "Linus", so the function will take it
```

```
The youngest child is Linus
```

```
my_function("Emil", "Natalie") # we don't have index 2
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call
last)
/tmp/ipykernel_726/555854529.py in <cell line: 0>()
----> 1 my_function("Emil", "Natalie")

/tmp/ipykernel_726/771745628.py in my_function(*kids)
      1 def my_function(*kids):
----> 2   print("The youngest child is " + kids[2]) # kids[2] mean
```

```
from the arguemnts in kids it will take the value at 2nd index

IndexError: tuple index out of range

def func(*myFamily):
    for i in myFamily: # we are iterating through the arguments
        print(i)
func('John', "Jessica", "Doe", "Justin", "staticmethod") # this are
the arguemnts/ you can add more if you want

John
Jessica
Doe
Justin
staticmethod
```

2. Keyword Arguments

- we can also send arguments with the key = value structure
- This way the order of the arguments does not matter.

```
def adder(a, b):
  return a + b

adder(3, 90)

93

def introduction(name, age):#name:str, age:int
  age += 10 #40
  print(f"Hello my name is {name}. I will be {age} years old in 10
years.")

introduction(age=30, name="Ejigayew") #name =30, age = "Ejigayew"

Hello my name is Ejigayew. I will be 40 years old in 10 years.

def my_function(child3, child2, child1):
  print("The youngest child is " + child3)

my_function(child1 = "Emil", child2 = "Tobias", child3 = "Linus") # as
you can see, we are explicitly giving value for each arguemnt as key-
value pair

The youngest child is Linus

def my_function(child3, child2, child1):
  print("The youngest child is " + child3)

my_function("Emil", "Tobias","Linus")

The youngest child is Emil
```

```python
def intro(name,age):
    print(f"My name is {name}. I am {age} is old.")
intro(age=7,name='Mihiret')
```

```
My name is Mihiret. I am 7 is old.
```

```python
def my_description(age, height, name):
    print("hello my name is:", name)
    print("I am ", age, "years old.")
    print("My height is", height, 'meters.')
my_description(name = "Asrat", age=22, height=1.72) # the order
doesn't matter if we using key-word arguments
```

```
hello my name is: Asrat
I am  22 years old.
My height is 1.72 meters.
```

```python
#be mindful, unless we are using key-word arguments, order of
arguments is important, look at the following example:
def calculate_BMI(name, weight, height):
    heightSquared = height**2
    BMI = weight/heightSquared
    print("Hello", name, "your BMI is", BMI)
calculate_BMI("Alem", 70, 1.72) # look now it works fine because
everything is in the right order even though we are not using key word
argument
```

```
Hello Alem your BMI is 23.661438615467823
```

```python
# let's mess up the argument order
calculate_BMI(1.72, "Alem", 70) # here now the orginal order is messed
up, this will cause an error
                                # python will take 1.72 as name,
"Mesay" as weight and 70 as height. which is wrong
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call
last)
/tmp/ipython-input-3880352591.py in <cell line: 0>()
      1 # let's mess up the argument order
----> 2 calculate_BMI(1.72, "Alem", 70) # here now the orginal order
is messed up, this will cause an error
      3                                 # python will take 1.72 as
name, "Mesay" as weight and 70 as height. which is wrong

/tmp/ipython-input-4000890748.py in calculate_BMI(name, weight,
height)
      2 def calculate_BMI(name, weight, height):
      3     heightSquared = height**2
----> 4     BMI = weight/heightSquared
```

```
      5     print("Hello", name, "your BMI is", BMI)
      6 calculate_BMI("Alem", 70, 1.72) # look now it works fine
because everything is in the right order even though we are not using
key word argument

TypeError: unsupported operand type(s) for /: 'str' and 'int'

calculate_BMI(height=1.72, name='Alem', weight=70) # now it works
fine, even though the original arugment order is not kept
                                                        # the key word
pair makes it specific

Hello Alem your BMI is 23.661438615467823
```

3. Default Parameter Value

- If we call the function without argument, it uses the default value

```
def my_function(country = "Ethiopia"): # we are specifying default
value when we construct the function itself
  print("I am from " + country)

my_function("Sweden")

I am from Sweden



my_function("India")

I am from India

my_function() # now we dont' have any argument, which means it takes
the default one

I am from Ethiopia

def intro(name, city="Durame"):
  print(f"Hello {name}, are you from {city}?")

intro("Tizita", "Hawassa")

Hello Tizita, are you from Hawassa?

intro("Tizita")

Hello Tizita, are you from Durame?
```

We can call a function inside another function

```
def addTwo(x):
    return x + 2  # This function adds 2 to the number and returns the
value
```

```
def mystery(x):
    y = addTwo(x)  # we are calling 'addTwo' function
    z= addTwo(y)
    return z

mystery(10) # Try to trace the code before running it
#myster(10) .. x = 10
  # y = addTwo(10) = 12
  # z= addTwo(12) = 14
```

```
14
```

```
def reverse(text):
  return text[::-1]

def isPalindrome(text):
  if text == reverse(text): # calling reverse function inside our
function
    return True
  return False

isPalindrome("level")
```

```
True
```

Variable Scoping

- Variables inside a function have no relation to variables outside the function
- This is complex topic... we will just cover the basics

```
# see what happens when 'i' is printed inside and outside the function
i = 0  # the value of i outside the function
def func(x):
    i = 10  # the value of i inside the function
    print(i, x)
func(5) # as you can see from the output, this one prints out i as 10
because the i inside the function is 10
print(i) # this one prints out i as 0, because outside the function i
is 0.
```

```
10 5
0
```

```
# Variables across functions have no relation to each other
# A variable declared inside a function will have no relation to
outside the function
def func1(x): #x=3
    j = 10
    print(j, x) #10, 3
def func2(x): #x=3
```

```
    j = 5
    func1(x) #3
    print(x) #3
func2(3)
print(j)
#why is there an error?
```

```
10 3
3

---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-3166485922.py in <cell line: 0>()
      9     print(x) #3
     10 func2(3)
---> 11 print(j)
     12 #why is there an error?

NameError: name 'j' is not defined
```

```
# what will be the output for the following code?
k = 0
for i in range(3): #0, 1, 2
    print(k) #0, 0, o
```

```
0
0
0
```

```
# what will be the output for the following code too
k = 0
for i in range(3):
    k = 6
    print(k)
```

Recursion

Recursion is when you call function inside the function itself.

Let's look at the following function that prints the given text given the numberOfTimes.

```
def repeatPrint(text, numberOfTimes):
  #your code here
```

For example:

```
repeatPrint("hello!", 3)
Should output:

Hello!
Hello!
Hello!
```

`implementation of repeatPrint using a for loop:`

```
def repeatPrint(text, numberOfTimes):
  for i in range(numberOfTimes):
    print(text)

#let's try it
repeatPrint("Hello!", 3)

Hello!
Hello!
Hello!
```

`implementation of repeatPrint using a while loop:`

```
def repeatPrint(text, numberOfTimes):
  i = 0
  while i < numberOfTimes:
    print(text)
    i+=1

#let's try it
repeatPrint("Hello!", 3)

Hello!
Hello!
Hello!
```

Let us now define repeatPrint using a technique called **Recursion**. *`A recursive function is a function that calls itself.`*

`Implementation of repeatPrint using recursion:`

```
def repeatPrint(text, numberOfTimes):
  if numberOfTimes == 0:
    #if the numberOfTimes to print is 0, there is no work to do.
    return
  print(text)
  #now we have one fewer number of times to print the text.
  numberOfTimesRemaining = numberOfTimes - 1
  repeatPrint(text, numberOfTimesRemaining) #here we are calling hte
function again with 1 number less
```

```
#let's try it:
repeatPrint("Hello!", 3)
```

```
Hello!
Hello!
Hello!
```

Every recursive function has two important parts: the **base case** and the **recursive case**.

The **base case** occurs when we have the simplest pssible input to the function, where the answer is obvious. `For repeatPrint the base occurs when numberOfTimes == 0`.

The **recursive case** occurs all other times from the **base case** where we are required to call the function itself with a different input(that gets one step closer to the base case). `For repeatPrint the recursive case occurs when numberOfTimes > 0.`

`What happens if we forget the base case?`

```
def repeatPrint(text, numberOfTimes):
  #print the text once
  print(text)
  #Now we hae one fewer number of times to print the text
  numberOfTimesRemaining = numberOfTimes - 1
  repeatPrint(text, numberOfTimesRemaining)

#let's try it:
repeatPrint("Hello!", 3)
```

```
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
```

```
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!
Hello!

---------------------------------------------------------------------------
RecursionError                            Traceback (most recent call last)
/tmp/ipython-input-1743060316.py in <cell line: 0>()
      7
      8 #let's try it:
----> 9 repeatPrint("Hello!", 3)

/tmp/ipython-input-1743060316.py in repeatPrint(text, numberOfTimes)
      4   #Now we hae one fewer number of times to print the text
      5   numberOfTimesRemaining = numberOfTimes - 1
----> 6   repeatPrint(text, numberOfTimesRemaining)
      7
      8 #let's try it:

... last 1 frames repeated, from the frame below ...

/tmp/ipython-input-1743060316.py in repeatPrint(text, numberOfTimes)
      4   #Now we hae one fewer number of times to print the text
      5   numberOfTimesRemaining = numberOfTimes - 1
----> 6   repeatPrint(text, numberOfTimesRemaining)
      7
      8 #let's try it:

RecursionError: maximum recursion depth exceeded while calling a Python object
```

Answer: repeatPrint will recursively all itself repeatedly, without stopping!! Similar to infinite loop!

Let's try out recursion with another example

The following sequene is called the Fibonacci sequence:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Can you the pattern in this sequence?

The **Fibonacci sequence** is defined as follows:

```
F(0) = 0
F(1) = 1
F(n) = F(n-1) + F(n-2), for n>=2
```

We can expand this out a bit to see the first few elements of this sequence:

```
F(0) = 0, by definition
F(1) = 1, by definition
F(2) = F(1) + F(0) = 1 + 0 = 1
F(3) = F(2) + F(1) = 1 + 1 = 2
F(4) = F(3) + F(2) = 2 + 1 = 3
F(5) = F(4) + F(3) = 3 + 2 = 5
. . .
```

This sequence apperas in algorithms, math, and in nature in various places. We will nt disucss the sequence itself in details, but you can read more about it here:

https://en.wikipedia.org/wiki/Fibonacci_sequence

Let's define a function **fib(n)** that computs the `nth Fibonacci number`

```
def fib(n):
  #code...
```

Let's implement it using loop first

```
def fib(n):
  if n == 0 or n == 1:
    return n

  fib_i_prev_prev = 0 # Initialized to F(0)
  fib_i_prev = 1      # Initialized to F(1)

  for i in range(n - 1):
    #Computer the number in the sequence
    fib_i = fib_i_prev_prev + fib_i_prev
    #save the state to use for the next computation
```

```
    fib_i_prev_prev = fib_i_prev
    fib_i_prev = fib_i

  return fib_i

#let's try it
fib(4)

3
```

`Implementation of fib(n) using recursion:`

```
def fib(n):
  #base case
  if n == 1 or n == 0:
    return n

  #recursive case
  return fib(n-1) + fib(n-2)


#check
fib(4)

3
```

Let's understand how recursive functions work with a couple of more examples

`What is the behaviour of this function?`

```
def mystery(a):
  if a == 0:
    return 0
  return a + mystery(a-1)

#trace and guess teh answer before running it
mystery(4)


#Answer
# mysters(a) = 0 + 1 + 2 ... + a
```

`How about the following function?`

```
def mystery2(a, b):
  if b == 0:
    return 0
```

```
  return a + mystery2(a, b-1)

#guess the answer first
mystery2(5, 4)

#Answer
#mystery2(a, b) = a*b
```

Practice Exercises

1. Write a Python function named `"isUnique"` that takes string as an argument and checks if all characters are unique. `returns True if it is unique and returns False otherwise.`

Example

```
isUnique("Amber")
answer: True #because all characters(letters) are used only once

isUnique("Banana")
answer: False #a and n are repeated more than once

def isUnique(word):
  pass
  #your code here
```

1. Write a function named `"circleArea"` that takes radius of circle and returns the area.

```
#your code here
```

1. Write a function named "Counter" that takes two attributes one list and anotehr value. Where it returns the number of times the value is repeated in the list.

Example:

```
l = [2, 3, 5, 2, 8, 9, 9, 2, 3, 2]
Counter(l, 2)
Answer: 4 #2 is repeated 4 times

Counter(l, 3)
Answeer: 2 #3 is repeated 2 times

#your code here
```

1. Write a function named `vowelCounter` tha takes a string and counts the number of vowels in the string.

Example:

```
vowelCounter("Ecuador")
Answer: 4 #E, u, a, o

#write your code here
```

1. Let's go back to our palindrome again. Write two functions:
- that takes string and reverses it and returns the result.
- that takes string and checks if the string is palindrome using the reverse function.

```
def reverse(word):
  pass
  #write your code here

def isPalindrome(word):
  pass
  #write your code here
```